Soil Moisture Monitoring System

Course: DATA 706 - Introduction to IoT – Février 2026 – Télécom Paris

Antoine Dalle & Guy Rostan
DJOUNANG NANA

Table of contents

1.Service Concept & IoT Value Proposition

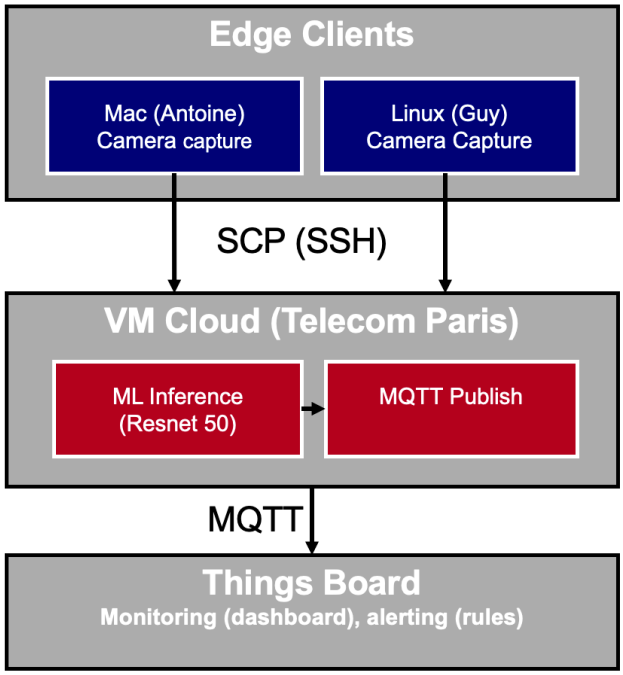
1.1 Problem & Vision

Traditional soil moisture monitoring **requires manual measurements**, is labor-intensive, and lacks real-time feedback. For agriculture and environmental monitoring, this leads to **water waste and delayed decision-making.**

We propose an **autonomous soil moisture monitoring** service based on computer vision. **Any camera-equipped device** (laptop, Raspberry Pi, smartphone) becomes a monitoring station. A machine learning model running on a **remote VM estimates** moisture from soil images and publishes results to a **dashboard via MQTT.**

Unlike traditional IoT moisture sensors (capacitive or resistive probes), this approach is non-invasive (no sensor in soil, no calibration drift, no corrosion) and scalable (adding a device requires only a camera and network access)..

1.2 IoT Architecture Overview



1.**Edge → VM:** Image transfer (heavy payload, ~300 KB–3 MB) over SCP/SSH
2.**VM → ThingsBoard:** Telemetry publication (lightweight JSON, ~200 bytes) over MQTT
3.**ThingsBoard → User:** Dashboard visualization, alerts, historical charts

1.3 Hypotheses on Operational Value

The following gains are not formally measured but represent plausible qualitative hypotheses.

Reduced manual effort: Automated hourly captures eliminate the need for on-site readings during the monitoring period.

Higher measurement frequency: Up to 24 data points per day versus the typical 1–3 manual readings, enabling intra-day trend detection.

Full traceability: Every prediction is timestamped and archived (image + JSON), providing audit trails that manual logs cannot match.

Consistent readings: Automated inference removes inter-operator variability in visual soil assessment.

1.4 Confidence Interval: Principle & Justification

The ML model outputs a moisture percentage with a 95% confidence interval of ±30.7%.

Principle & Width. The interval is derived from the 95th percentile of errors during training. It is currently wide because the model operates on generic training data without site-specific calibration.
Value.

This interval provides necessary transparency, preventing false alerts based on single readings. For decision-making, the trend over time is more reliable than the absolute point estimate.
Reduction Requirement.

Narrowing this interval is possible but requires a significant Data Science investment: specifically, collecting site-specific ground truth data to re-train and fine-tune the model (see Section 4).

2. Protocol Architecture:

2.1 System Components

Edge layer (2 laptops):

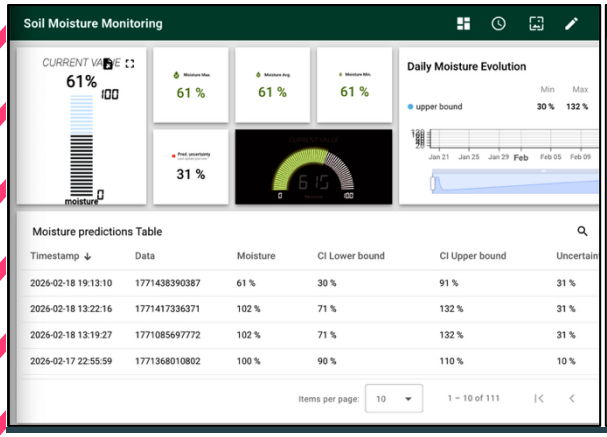
Component	Choice	Rationale
OS	macOS 14 / Ubuntu 22.04	Cross-platform validation
Capture tool	imagesnap (Mac) / fswebcam (Linux)	Lightweight, scriptable, no GUI
Transfer	SCP over SSH	Encrypted, reliable, universally available
Scheduling	Bash loop with sleep	Simple, no extra dependency

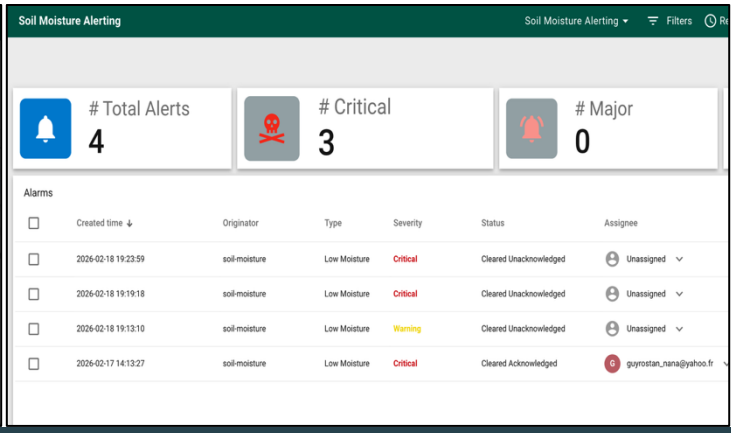
Cloud layer (1 VM):

Component	Choice	Rationale
VM	tp-hadoop-12 (Télécom Paris)	Free SSH access, representative of cloud VMs
ML inference	ResNet-50 features + Ridge regression	Lightweight CPU model (9.4 KB), no GPU required
MQTT client	paho-mqtt (Python)	Standard client, ThingsBoard-compatible

A lightweight CPU model was chosen specifically to enable inference on a standard VM without GPU.

Dashboard layer (monitoring & alerting):





2.1 Data Flow

The pipeline executes in four sequential steps, fully automated after initial setup:

Step 1 — Capture (Edge). A bash script runs in a loop, triggering the camera every hour. Each capture produces a timestamped JPEG image stored locally.

Step 2 — Transfer (Edge → VM). The image is transferred to the VM via SCP. Authentication uses SSH key pairs; the channel is encrypted (AES-256). Typical transfer time: ~2.3s for a 1 MB image.

Step 3 — Inference + Publish (VM). The inference script loads the ML model, extracts visual features, predicts moisture with confidence bounds, and **immediately publishes** the result to ThingsBoard via MQTT. A local JSON backup is also saved for audit. This is a single script execution — there is no separate watcher or manual handover step.

Step 4 — Visualization (ThingsBoard). The dashboard updates in real-time. Users see the latest moisture reading, historical trend, and any triggered alerts.

End-to-end latency: ~3s (capture: 0.5s, transfer: 0.5s, inference: 1s, MQTT publish: 0.5s).

2.2 Storage on VM

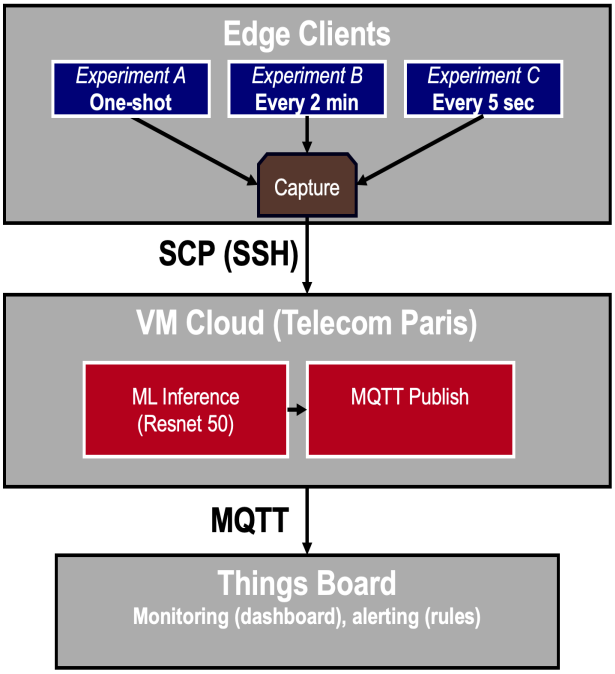
```
~/soil_moisture/
├── incoming/ # Raw images (temporary)
├── processed/ # Archived images
├── logs/predictions/  # JSON backups
├── models/  # ML model artifact
└── src/ # Inference + MQTT script
```

3. Experimental Validation

3.1 Experimental Design

The experimental validation aims to assess the **end-to-end reliability and robustness of the IoT pipeline** under three operational conditions: **single-shot execution**, **stable periodic monitoring**, and **burst load**. The goal is not to evaluate ML accuracy per se, but to observe how the full chain — capture, transfer, inference, MQTT publish, and dashboard update — behaves across varying conditions.

3.2 Experiment Architecture



3.3 Three Operational Experiments

Three scripts were designed to cover the main operational scenarios of the pipeline, from basic validation to stress testing.

Experiment A — One-Shot Capture (`capture_and_predict.sh`)

A single image is captured, transferred to the VM via SCP, processed by the inference script, and the result is published to ThingsBoard via MQTT. This experiment validates the basic end-to-end pipeline and serves as the reference for latency measurement.

Rationale: Confirms that all components (camera, SSH tunnel, inference, MQTT broker, dashboard) are correctly connected and functional before running longer experiments.

Experiment B — Stable Interval Test (`auto_capture_loop.sh`, 2-minute interval)

The capture loop runs continuously, taking one image every 2 minutes over a period of 1–2 hours. Each iteration follows the same A→B→C→D chain as Experiment A.

Rationale: Tests whether the pipeline maintains stable behavior over repeated executions. Key observations: Does latency drift? Are there accumulating failures? Does ThingsBoard show a continuous time-series without gaps?

Experiment C — Burst / Stress Test (`auto_capture_loop.sh`, 5-second interval)

The capture interval is reduced to 5 seconds for a short period (10–15 minutes), generating a high volume of sequential requests to the VM inference stack and the MQTT broker.

Rationale: Stresses the pipeline to detect bottlenecks — SCP queue congestion, inference backlog on VM CPU, MQTT publish failures, or ThingsBoard rate-limiting. Results from this experiment directly motivate the To-Be improvements (Section 4).

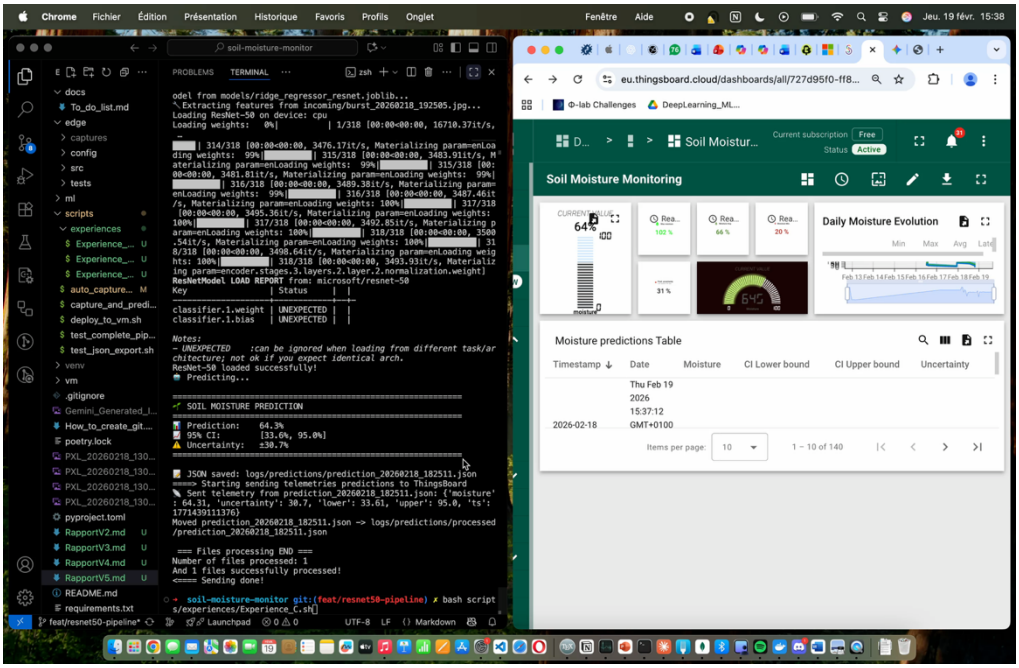
3.4 Results

All three experiments completed successfully. The pipeline demonstrated full end-to-end reliabili across single-shot, stable interval, and burst conditions.

Metric	Exp. A — One-Shot	Exp. B — 2 min interval	Exp. C — Burst 5 sec
Capture success rate	**100%**		
SCP transfer success rate			
Inference success rate			
MQTT publish success rate			
Missing points on ThingsBoard	**None**		
End-to-end latency	**Stable, no drift**		
Anomaly observed	**None**		

Experiment C outlier: The first capture returned an atypical prediction 24%, confirming the model's sensitivity to framing and validates the importance of camera placement *(see video)*

3.5 Video Demonstration



Experiment C: Terminal Ouput & Thingsboard time-series dashboard during the burst test (5-second interval).

4. Improvements, Limitation & Conclusion

4.1 Improvements

#	Improvement	IoT Benefit
1	SCP timeout (15s connection limit)	Prevents indefinite hangs on network failure
2	SCP retry with exponential backoff (3 retries: 10s, 20s, 30s)	Recovers from transient failures without manual intervention
3	Local buffering on edge (store unsent images in buffer/, flush on recovery)	Zero data loss during VM or network outages
4	MQTT QoS 1 (at-least-once delivery)	Broker acknowledges receipt; eliminates silent publish failures
5	Message idempotence via message_id	Prevents duplicate processing if retry delivers the same message twice
6	Multi-device support via device_id field in JSON payload	Enables per-device filtering on ThingsBoard; required to scale beyond 2 clients
7	JSON schema versioning ("schema_version": "1.0")	Future-proofs the payload format; ThingsBoard rules can filter by version
8	Log rotation + basic monitoring	Prevents disk saturation on VM; detects silent pipeline failures early
9	SQLite database on the VM (queryable history)	Useful for retry on failed deliveries

This section proposes incremental improvements directly motivated by the limitations observed in the current prototype and anticipated from experiments.

4.2 Limitations

1. **No network resilience:** SCP has no timeout or retry. Any network interruption causes silent, unrecoverable data loss in the current iteration.
2. **No local buffering:** Captured images that cannot be transferred are not queued. The edge device has no fallback mechanism.
3. **Laptop sleep mode:** In the 24-hour test, one capture was missed due to the Mac entering sleep mode. This is a hardware constraint to take into consideration in the IoT devices choices.
4. **Hardware mismatch:** The model was trained on a Mac (MPS GPU) but deployed on a CPU-only VM.
5. **Wide confidence interval (±30.7%):** Acceptable for trend-based irrigation decisions but insufficient for precision use cases. Root cause: coarse training labels and absence of site-specific calibration.
6. **Flat file storage:** JSON logs accumulate without structure or rotation. No queryable history and no publish-status tracking.

4.3 Conclusion

The prototype implements an automated pipeline edge capture across two platforms (macOS and Linux), SCP transfer, CPU-based ML inference, MQTT telemetry publication, & real-time ThingsBoard visualization with no manual intervention after initial setup.

The main weaknesses are the absence of network resilience, no local buffering, and a wide confidence interval.

The prototype validates the core IoT architecture.

The proposed incremental improvements represent realistic next steps

that would bring the system to pilot-ready reliability for 5–10 edge devices, without requiring enterprise-grade infrastructure.

Repository:

GitHub — soil-moisture-monitor

Contact:

antoine.dalle@.telecom-paris.fr

guy.djounangnana@telecom-paris.fr